

Heverton Eduardo Peres

Subagentes: Fan-out Paralelo de Tarefas & O Advisor Model: Um Revisor a Cada Turno

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

Brasil
2026

Sumário

Subagentes: Fan-out Paralelo de Tarefas	3
Escalando para múltiplos workers	3
O que são subagentes e por que existem	3
O sistema de tasks do OMP	3
Schema validation: resultados tipados	4
O Agent Hub: painel de controle central	4
O ciclo de vida de uma task	5
Fan-out/Fan-in: o padrão clássico	5
Criando sua primeira task	5
Spawning um subagente	6
Fan-out com múltiplos workers	6
Collecting resultados	7
Caso de uso: code review paralelo	8
Caso de uso: migração de framework	8
Caso de uso: refactoring distribuído	8
Próximos Passos	8
O Advisor Model: Um Revisor a Cada Turno	10
O inspetor de casco digital	10
O que é um Advisor Model	10
Como o advisor observa e injeta notas	11
Configuração de modelo advisor separado	11
Protocolo de decisão	11
A metáfora do estaleiro	12
Configurando seu primeiro advisor model	12
Integração com o agente principal	13
A cena de contraste	14
Armadilhas comuns do advisor model	14
Métricas de sucesso com advisor	14
Próximos Passos	15

Subagentes: Fan-out Paralelo de Tarefas

Escalando para múltiplos workers

No capítulo anterior, você dominou o debug com DAP — 28 operações que transformam o agente em um debugger autônomo capaz de pausar processos, inspecionar variáveis e corrigir bugs na raiz.

Agora vamos escalar: em vez de um único agente fazendo tudo sequencialmente, vamos lançar múltiplos workers em paralelo — cada um isolado, com sua própria memória e resultado tipado.

O que são subagentes e por que existem

Quando você trabalha com um coding agent sozinho, existe um limite natural de produtividade: o agente pode fazer apenas uma coisa por vez. Se sua tarefa requer analisar 50 arquivos de código, revisar 10 pull requests ou migrar um projeto inteiro de uma framework para outro, um único agente vai levar tempo demais — e o contexto dele vai encher antes de terminar.

Subagentes resolvem esse problema dividindo o trabalho. Cada subagente é um worker isolado que recebe uma tarefa específica, executa de forma autônoma e retorna um resultado tipado.

É como transformar um único estaleiro com um único operador em um estaleiro com múltiplas equipes especializadas trabalhando em paralelo.

O sistema de tasks do OMP

O OMP implementa subagentes através da ferramenta `task`, que suporta três operações principais.

create — cria uma nova tarefa com um ID único (T1, T2, T3...) e um resumo do que precisa ser feito.

spawn — lança um subagente para executar a tarefa, que roda em background e retorna um `actor_id`.

collect — aguarda o resultado do subagente e valida seu output contra um schema.

A hierarquia de tasks permite criar estruturas complexas: uma tarefa pai pode ter múltiplas subtarefas (T1.1, T1.2, T1.3), cada uma com seu próprio subagente. Essa é a base do fan-out paralelo — lançar vários workers simultaneamente para cobrir diferentes aspectos de um problema.

Schema validation: resultados tipados

Uma das features mais poderosas do sistema é a validação de schema. Quando você cria uma task, pode definir um `output_schema` que especifica exatamente quais campos o subagente deve retornar.

Isso garante que cada subagente retorna exatamente o que foi pedido, erros de formato são detectados antes de você processar o resultado, e múltiplos subagentes podem ser comparados diretamente porque seguem o mesmo formato.

É como se cada equipe do estaleiro tivesse um formulário padrão de reporte. Sem essa padronização, você teria que interpretar relatórios livres de cada especialista.

O Agent Hub: painel de controle central

Enquanto os subagentes trabalham em paralelo, o Agent Hub fornece visibilidade total sobre o que está acontecendo. Você pode ver quais subagentes estão rodando, pendentes ou completados, monitorar o progresso de cada um em tempo real, cancelar subagentes que estão demorando ou falhando, e ajustar timeouts e limits de recursos.

O Agent Hub é o equivalente ao painel de controle do estaleiro — onde o mestre vê todas as equipes trabalhando e pode intervir quando necessário.

O ciclo de vida de uma task

Cada task passa por um ciclo de vida bem definido. Ela começa como `open`, vai para `in_progress` quando um worker é spawned, e pode terminar como `done` ou `failed`. Se uma tarefa depender de outra, ela fica `blocked` até a dependência ser resolvida.

Esse ciclo garante que você sempre saiba o estado de cada tarefa. Se um worker falhar, a task volta para `open` e pode ser retomada.

Fan-out/Fan-in: o padrão clássico

O padrão mais comum de subagentes é o fan-out/fan-in.

Primeiro, você distribui uma tarefa grande entre múltiplos workers — por exemplo, revisar 10 arquivos diferentes. Depois, cada worker processa sua parte independentemente. Por fim, você coleta todos os resultados e os consolida.

É como um estaleiro que distribui a construção do navio entre equipes especializadas — cada uma monta sua parte em paralelo, e no final todas as peças se encaixam perfeitamente porque seguem o mesmo schema de fabricação.

Criando sua primeira task

A ferramenta `task` é a porta de entrada para subagentes.

```
# Exemplo: criar uma task para revisar um arquivo
task_create = {
    "tool": "task",
    "input": {
        "action": "create",
        "summary": "Revisar o arquivo main.py e identificar bugs",
        "output_schema": {
            "type": "object",
            "properties": {
                "bugs_encontrados": {"type": "array", "items": {"type":
"string"}}},
                "severidade": {"type": "string", "enum": ["baixa", "media",
"alta"]},
                "recomendacoes": {"type": "array", "items": {"type":
"string"}}}
            }
        }
    }
```

```

        },
        "required": ["bugs_encontrados", "severidade"]
    }
}
}

```

Quando você envia essa task, o OMP retorna um ID (ex: T1) que você usa para rastrear o progresso.

Spawning um subagente

Depois de criar a task, você lança um subagente para executá-la.

```

# Exemplo: spawn um subagente para a task T1
task_spawn = {
    "tool": "task",
    "input": {
        "action": "spawn",
        "task_id": "T1",
        "prompt": "Analise o arquivo main.py. Procure por: bugs lógicos,
erros de tratamento de exceção, código morto, e potential race
conditions.",
        "context": "full"
    }
}

```

O subagente roda em background e retorna um `actor_id` que você usa para monitorar seu progresso.

Fan-out com múltiplos workers

Para distribuir trabalho entre vários workers, crie múltiplas tasks e spawne um subagente para cada uma.

```

# Exemplo: fan-out para revisar 3 arquivos diferentes
files_to_review = ["src/auth.py", "src/api.py", "src/models.py"]

for i, file in enumerate(files_to_review, 1):
    task_id = f"T{i}"
    # Criar task

```

```

task_create = {
    "tool": "task",
    "input": {
        "action": "create",
        "task_id": task_id,
        "summary": f"Revisar {file} para bugs e más práticas"
    }
}

# Spawn worker
task_spawn = {
    "tool": "task",
    "input": {
        "action": "spawn",
        "task_id": task_id,
        "prompt": f"Analise o arquivo {file}. Identifique bugs, código
morto, e oportunidades de refactoring."
    }
}

```

Agora três workers estão rodando em paralelo, cada um analisando um arquivo diferente.

Collecting resultados

Quando todos os workers terminarem, você coleta os resultados.

```

# Exemplo: aguardar e coletar resultados
task_collect = {
    "tool": "task",
    "input": {
        "action": "collect",
        "task_ids": ["T1", "T2", "T3"],
        "timeout_ms": 60000
    }
}

# Resultado consolidado
resultado = {
    "T1": {"bugs": ["SQL injection em login"], "severidade": "alta"},
    "T2": {"bugs": ["Race condition em /api/users"], "severidade":
"media"},
    "T3": {"bugs": [], "severidade": "baixa"}
}

```

O Agent Hub coleta todos os resultados e os apresenta de forma consolidada.

Caso de uso: code review paralelo

Imagine que você tem um pull request com 10 arquivos modificados e precisa revisar todos antes de mergear. Em vez de revisar sequencialmente (o que levaria 30+ minutos), você distribui entre 5 workers.

Em vez de 30 minutos sequenciais, você resolve em 5 minutos paralelos — 6x mais rápido.

Caso de uso: migração de framework

Migrar um projeto de uma framework para outra é trabalhoso e propenso a erros. Subagentes tornam isso gerenciável.

Cada worker opera isolado em seu módulo — não há conflitos de edição porque cada um trabalha em arquivos diferentes.

Caso de uso: refactoring distribuído

Refactoring em grande escala pode ser assustador. Subagentes permitem dividir o trabalho entre especialistas.

Cada worker foca em um tipo específico de melhoria — type hints, logging, testes — e todos trabalham em paralelo sem conflitos.

Próximos Passos

Neste capítulo, você aprendeu a transformar um único agente em uma equipe inteira de workers paralelos. O sistema de subagentes do OMP oferece três capacidades fundamentais.

Fan-out com schema validation: você distribui trabalho entre múltiplos workers, cada um retornando resultados tipados que garantem consistência e facilitam consolidação.

Agent Hub para monitoramento ao vivo: você mantém visibilidade total sobre seus subagentes — sabendo quando intervir, quando cancelar e quando coletar resultados.

Padrões de uso prático: code review paralelo, migração de framework e refactoring distribuído são apenas alguns dos casos de uso que se beneficiam do fan-out paralelo.

No próximo capítulo, veremos como o Advisor Model funciona como um revisor a cada turno — um segundo olhar que garante qualidade antes que cada peça seja montada no casco.

Acesse a documentação completa: <https://omp.sh/docs>

Siga-nos nas redes sociais para dicas, tutoriais e novidades sobre o Oh My Pi.

O Advisor Model: Um Revisor a Cada Turno

O inspetor de casco digital

No capítulo anterior, você dominou sub-agentes: aprendeu a criar tarefas com `task`, a spawnar workers, a coordenar execução paralela e a construir workflows completos. O agente agora delega — mas delegar tarefas não é o mesmo que receber feedback sobre seu próprio trabalho.

Existe uma diferença fundamental entre dizer “faça isso para mim” e ouvir “olha, o que você fez tem um problema aqui”. O advisor model é exatamente essa segunda capacidade: um segundo modelo LLM que observa cada turno do agente principal e emite notas qualificadas sobre a qualidade, segurança e coerência do que está sendo produzido.

O que é um Advisor Model

Um advisor model é um segundo LLM configurado especificamente para revisar o trabalho do agente principal. Enquanto o agente principal foca em executar tarefas (ler código, editar arquivos, rodar comandos), o advisor foca em avaliar a qualidade, segurança e coerência do que está sendo feito.

A mecânica é simples: a cada turno, o agente principal envia seu contexto atual para o advisor. O advisor analisa esse contexto e retorna notas estruturadas — preocupações (concerns), bloqueios (blockers) ou sugestões (suggestions). O agente principal então decide se incorpora essas notas ao seu trabalho ou as descarta.

Essa separação de papéis é fundamental. O agente principal é o operário do estaleiro — ele constrói, edita, executa. O advisor é o inspetor — ele verifica, aponta, recomenda. Nenhum dos dois faz o trabalho do outro.

Como o advisor observa e injeta notas

O advisor recebe três tipos de informação: o histórico de turnos (o que o agente já fez), o plano atual (o que o agente vai fazer agora) e o código afetado (os arquivos que serão modificados). Com base nesses dados, o advisor emite notas em formato estruturado.

Concern (preocupação): algo que não bloqueia a execução, mas que merece atenção. Exemplo: “A função `processar_pedido` não valida entrada — pode causar bugs em dados inválidos”. O agente pode decidir corrigir agora ou registrar para depois.

Blocker (bloqueio): algo que impede a continuação segura. Exemplo: “Este comando vai deletar o arquivo de configuração de produção”. O agente DEVE parar e resolver o blocker antes de prosseguir.

Suggestion (sugestão): melhoria opcional que aumenta qualidade. Exemplo: “Considere adicionar um try/except aqui — esta operação pode falhar em rede instável”. O agente decide se incorpora.

Configuração de modelo advisor separado

A escolha do modelo advisor é estratégica. O advisor precisa de três características que o agente principal pode não ter: foco em análise crítica, conhecimento de padrões de segurança e qualidade, e capacidade de emitir julgamentos concisos.

Um modelo grande e geral (como Claude Opus ou GPT-4o) funciona bem como agente principal, mas um modelo menor e mais focado (como Claude Haiku ou GPT-4o-mini) pode ser mais eficiente como advisor — porque o advisor não precisa gerar código, apenas avaliar.

Protocolo de decisão

A integração entre agente principal e advisor segue um protocolo definido. O agente não é obrigado a obedecer ao advisor — ele é obrigado a CONSIDERAR o que o advisor diz. Essa distinção é crucial: o advisor é um consultor, não um comandante.

O protocolo de decisão funciona assim.

Blocker recebido: o agente PARA. Analisa o blocker. Se concorda, corrige o problema. Se discorda, registra a justificativa e continua — mas o registro fica no log para auditoria futura.

Concern recebido: o agente ANOTA. Continua trabalhando, mas mantém o concern em mente. Pode resolver agora ou deixar para depois — dependendo da prioridade.

Suggestion recebida: o agente AVALIA. Se a suggestion melhora significativamente o código, incorpora. Se não, ignora. Não há obrigação de aceitar suggestions.

O valor do advisor não está em impedir erros — está em REDUZIR a taxa de erros que passam despercebidos. Estudos mostram que agentes com advisor reduzem bugs de segurança em 40-60% comparados a agentes sem revisão.

A metáfora do estaleiro

Imagine seu estaleiro digital. Você é o Mestre de Estaleiro — responsável por construir navios. Cada navio tem um casco, um motor, equipamento de navegação e uma tripulação. Você coordena tudo, toma decisões rápidas, ajusta rotas quando necessário.

Mas existe um problema: quando você está construindo o casco, suas mãos estão ocupadas soldando. Quando está calibrando o motor, seus olhos estão no gauge de pressão. Você não consegue, ao mesmo tempo, CONSTRUIR e VERIFICAR.

É aí que entra o inspetor de casco — o advisor model. O inspetor não solda, não calibra, não navega. Ele anda pelo estaleiro, examina cada trabalho que você faz e aponta: “essa solda está fraca”, “essa viga está torta”, “esse cabo está solto”. Você decide se para para corrigir ou continua — mas o inspetor garante que você SABE dos problemas antes de zarpar.

Configurando seu primeiro advisor model

Para configurar um advisor model no Oh My Pi, você precisa de dois arquivos: a configuração do modelo advisor e o hook que integra o advisor ao fluxo do agente principal.

```
# Arquivo: .ohmypi/advisor-config.yaml

advisor:
  model: "claude-haiku-4-20250414"
  temperature: 0.1

  system_prompt: |
    Você é um revisor de código especializado em segurança e qualidade.
    Sua única tarefa é analisar ações do agente e emitir notas
    estruturadas.

  Regras:
    - SEMPRE emite BLOCKER quando detectar risco de segurança
    - SEMPRE emite BLOCKER quando detectar destruição de dados
```

- Emite CONCERN quando detectar violação de padrão
- Emite SUGGESTION quando detectar melhoria significativa
- NÃO emite nota para estilo ou formatação

```
thresholds:
  blocker: 0.9
  concern: 0.7
  suggestion: 0.5
```

Integração com o agente principal

A integração usa o hook `pre_tool_call` do Oh My Pi. O hook intercepta cada chamada de ferramenta antes que ela seja executada, envia o contexto para o advisor e processa a nota recebida.

```
@hook("pre_tool_call")
def advisor_hook(tool_name: str, arguments: dict, context: dict) -> dict:
    advisor_context = {
        "turno": context.get("turn_count", 0),
        "ferramenta": tool_name,
        "argumentos": arguments,
        "codigo_afetado": context.get("affected_files", []),
        "plano": context.get("current_plan", "N/A")
    }

    resultado = call_model(
        model="claude-haiku-4-20250414",
        prompt=advisor_prompt,
        temperature=0.1
    )

    nota = json.loads(resultado.get("content", '{"tipo": "ok"}'))

    if nota["tipo"] == "blocker":
        return {"action": "deny", "reason": f"Advisor bloqueou: {nota['mensagem']}"}
    elif nota["tipo"] == "concern":
        context.setdefault("advisor_concerns", []).append(nota)
        return {"action": "allow", "advisor_note": nota}
    else:
        return {"action": "allow"}
```

A cena de contraste

Imagine que você está desenvolvendo uma API de e-commerce. O agente principal está criando a rota de checkout — uma das partes mais críticas do sistema.

Sem advisor, o agente gera o código rapidamente: uma função que recebe o pedido, debita o estoque, cobra o cartão e envia o e-mail de confirmação. O código funciona nos testes. Você faz deploy. Na primeira compra real, o cartão é cobrado, mas o estoque não é debitado — porque o agente esqueceu de colocar a operação dentro de uma transação atômica.

Agora imagine o mesmo cenário COM advisor. O agente gera a mesma função. Antes de executar, o advisor analisa e emite um blocker: “A operação de débito de estoque e cobrança do cartão não está em uma transação atômica. Se uma falhar, a outra fica órfã.” O agente para, analisa o blocker, concorda e envolve a operação em uma transação bancária. O bug é pego ANTES do deploy.

A diferença não é o código gerado — é o MOMENTO em que o problema é detectado. Sem advisor, o bug aparece em produção. Com advisor, o bug aparece no estaleiro, quando ainda é barato corrigir.

Armadilhas comuns do advisor model

Confundir advisor com substituto humano. O advisor é uma camada adicional de defesa, não uma garantia. Ele reduz a taxa de erros, não a elimina.

Ignorar o advisor quando ele discorda. Se você ignora todos os blockers do advisor, por que configurou um advisor?

Não ter plano B. Quando o advisor trava (modelo indisponível, timeout), o agente precisa continuar funcionando, não parar tudo.

Configurar o advisor errado. Um advisor com regras muito restritivas bloqueia tudo; um advisor com regras muito permissivas não detecta nada. A calibração é chave.

Métricas de sucesso com advisor

Um time que usa advisor model se mede por quatro indicadores.

Taxa de bloqueios por advisor — quantas vezes o advisor bloqueou uma ação perigosa (meta: 5-15% das ações).

Tempo médio de resolução de blocker — quanto tempo o agente leva para resolver um blocker (meta: < 2 turnos).

Redução de bugs em produção — comparar antes e depois do advisor (meta: 30-50% de redução).

Custo do advisor — tokens consumidos pelo advisor em relação ao agente principal (meta: $< 20\%$ do custo total).

Próximos Passos

Neste capítulo, você adicionou uma camada fundamental de qualidade ao seu agente: o advisor model. Entendeu como o advisor observa cada turno e emite notas estruturadas — concerns, blockers e suggestions. Configurou um modelo advisor separado com system prompt especializado e thresholds de confiança. E dominou o protocolo de decisão — quando obedecer ao advisor, quando discordar, quando escalar.

O advisor model não é um luxo — é uma necessidade em projetos que usam agentes autônomos. Um agente sem advisor é como um navio sem inspetor de casco: pode zarpar, mas não há garantia de que vai chegar ao porto.

No próximo capítulo, vamos trazer um colega sentado ao seu lado no estaleiro digital: colaboração ao vivo com /collab.

Acesse a documentação completa: <https://omp.sh/docs>

Siga-nos nas redes sociais para dicas, tutoriais e novidades sobre o Oh My Pi.
